

Type Inference and Typing Rules in Haskell

Prepared Notes by Pushpendra

Contents

1	Introduction	2
2	Core Idea	2
3	Typing Context	2
4	Fundamental Typing Rules	2
4.1	Variable Rule (Var)	3
4.2	Abstraction (Lambda Rule)	3
4.3	Application (App)	3
4.4	If Expression	3
4.5	Let Binding	4
4.6	Constants	4
5	Derived Typing Rules	4
5.1	Pair Formation	4
5.2	Projection	4
5.3	List Rules (Haskell Specific)	4
6	Polymorphism	4
6.1	Generalization	5
6.2	Instantiation	5
7	Subtyping	5
8	Example: Step-by-step Type Inference	5
8.1	Example 1: Identity Function	5
8.2	Example 2: Function Composition	6
8.3	Example 3: Conditional	6

9 Where Type Inference Is Used	6
--------------------------------	---

10 Key Takeaways	6
------------------	---

1 Introduction

Type inference is the process by which a compiler automatically deduces the types of expressions without explicit type annotations. Haskell uses the **Hindley–Milner** type system, which guarantees that every well-typed expression has a **principal type** — the most general type from which all others can be derived by substitution.

Type inference makes Haskell programs concise while maintaining strong static type safety.

2 Core Idea

The goal of type inference is to derive a judgment of the form:

$$\Gamma \vdash t : T$$

which reads as: “*Under typing context Γ , term t has type T .*”

Example:

$$\begin{aligned} &\text{For } f\ x = x + 1, \text{ we infer:} \\ (+) &:: \text{Num } a \Rightarrow a \rightarrow a \rightarrow a \\ &\Rightarrow f :: \text{Num } a \Rightarrow a \rightarrow a \end{aligned}$$

3 Typing Context

The context Γ is a collection of variable-type associations:

$$\Gamma = \{x_1 : T_1, x_2 : T_2, \dots\}$$

It represents the environment in which type inference is performed.

4 Fundamental Typing Rules

Each typing rule expresses how to infer a type judgment. Below, Γ represents the typing environment.

4.1 Variable Rule (Var)

$$\overline{\Gamma, x : T \vdash x : T}$$

A variable has the type given to it in the context.

Example: If $\Gamma = \{x : Int\}$, then $\Gamma \vdash x : Int$.

4.2 Abstraction (Lambda Rule)

$$\frac{\Gamma, x : T_1 \vdash t : T_2}{\Gamma \vdash \lambda x. t : T_1 \rightarrow T_2}$$

A function is typed as $T_1 \rightarrow T_2$ if its body t has type T_2 under the assumption that $x : T_1$.

Example: For $\lambda x. x + 1$, since $x : Int$, we infer:

$$\Gamma \vdash \lambda x. x + 1 : Int \rightarrow Int$$

4.3 Application (App)

$$\frac{\Gamma \vdash t_1 : T_1 \rightarrow T_2 \quad \Gamma \vdash t_2 : T_1}{\Gamma \vdash t_1 t_2 : T_2}$$

If t_1 is a function and t_2 matches its input type, the result has type T_2 .

Example: For $(\lambda x. x + 1) 3$:

$$\Gamma \vdash (\lambda x. x + 1) : Int \rightarrow Int, \quad \Gamma \vdash 3 : Int \Rightarrow \Gamma \vdash (\lambda x. x + 1) 3 : Int$$

4.4 If Expression

$$\frac{\Gamma \vdash t_1 : Bool \quad \Gamma \vdash t_2 : T \quad \Gamma \vdash t_3 : T}{\Gamma \vdash \text{if } t_1 \text{ then } t_2 \text{ else } t_3 : T}$$

The condition must be Boolean, and both branches must have the same type.

Example:

$$\text{if True then 5 else 6} :: Int$$

4.5 Let Binding

$$\frac{\Gamma \vdash t_1 : T_1 \quad \Gamma, x : T_1 \vdash t_2 : T_2}{\Gamma \vdash \text{let } x = t_1 \text{ in } t_2 : T_2}$$

Used to define local bindings. In Haskell, it also supports polymorphism.

—

4.6 Constants

$$\Gamma \vdash n : \text{Int}, \quad \Gamma \vdash \text{True} : \text{Bool}, \quad \Gamma \vdash \text{False} : \text{Bool}$$

—

5 Derived Typing Rules

Derived rules are not primitive but can be built from the core ones.

5.1 Pair Formation

$$\frac{\Gamma \vdash t_1 : T_1 \quad \Gamma \vdash t_2 : T_2}{\Gamma \vdash (t_1, t_2) : T_1 \times T_2}$$

5.2 Projection

$$\frac{\Gamma \vdash t : T_1 \times T_2}{\Gamma \vdash \text{fst } t : T_1} \quad \frac{\Gamma \vdash t : T_1 \times T_2}{\Gamma \vdash \text{snd } t : T_2}$$

5.3 List Rules (Haskell Specific)

$$\Gamma \vdash [] : [a] \quad \frac{\Gamma \vdash x : a \quad \Gamma \vdash xs : [a]}{\Gamma \vdash x : xs : [a]}$$

—

6 Polymorphism

Haskell's type inference supports parametric polymorphism through **generalization** and **instantiation**.

6.1 Generalization

If a term t has type T and no type variables are fixed by Γ , we can generalize:

$$\frac{\Gamma \vdash t : T}{\Gamma \vdash t : \forall a. T}$$

Example:

$$id \ x = x \Rightarrow id :: \forall a. a \rightarrow a$$

6.2 Instantiation

$$\frac{\Gamma \vdash t : \forall a. T}{\Gamma \vdash t : [U/a]T}$$

Replace a with a concrete type U when using the polymorphic function.

Example:

$$id \ 3 : Int, \quad id \ True : Bool$$

—

7 Subtyping

Subtyping allows flexible type compatibility. The main rule is:

$$\frac{\Gamma \vdash t : S \quad S \leq T}{\Gamma \vdash t : T}$$

Function subtyping:

$$\frac{T'_1 \leq T_1 \quad T_2 \leq T'_2}{T_1 \rightarrow T_2 \leq T'_1 \rightarrow T'_2}$$

(Contravariant in input, covariant in output.)

—

8 Example: Step-by-step Type Inference

8.1 Example 1: Identity Function

$$id = \lambda x. x$$

Assign $x : \alpha$, then the body has type α . So:

$$\Gamma \vdash id : \alpha \rightarrow \alpha$$

Generalize over α :

$$id :: \forall a. a \rightarrow a$$

—

8.2 Example 2: Function Composition

$$\text{compose} = \lambda f. \lambda g. \lambda x. f(gx)$$

Assign:

$$f : \beta \rightarrow \gamma, \quad g : \alpha \rightarrow \beta, \quad x : \alpha$$

Then:

$$g \ x : \beta \quad f(g \ x) : \gamma$$

Hence:

$$\text{compose} : (\beta \rightarrow \gamma) \rightarrow (\alpha \rightarrow \beta) \rightarrow \alpha \rightarrow \gamma$$

Generalized:

$$\text{compose} :: \forall a \ b \ c. (b \rightarrow c) \rightarrow (a \rightarrow b) \rightarrow a \rightarrow c$$

8.3 Example 3: Conditional

$$f \ x = \text{if } x > 0 \text{ then } x \text{ else } (-x)$$

$$(>) :: \text{Ord } a \Rightarrow a \rightarrow a \rightarrow \text{Bool}$$

Hence:

$$f :: (\text{Ord } a, \text{Num } a) \Rightarrow a \rightarrow a$$

9 Where Type Inference Is Used

- **Haskell compiler (GHC)** uses it to deduce types automatically.
 - **Type checking** in functional languages (OCaml, F#, Scala).
 - **Static analysis tools** to ensure type safety before runtime.
 - **Type-driven optimizations** (specialization, unboxing).
 - **Generic programming** via polymorphism.
-

10 Key Takeaways

- Every expression has a most general type (principal type).
- Type inference proceeds by unification of type equations.
- Hindley–Milner ensures type safety without explicit annotations.

- Subtyping introduces flexibility but adds complexity to inference.

Summary

$$\Gamma \vdash t : T$$

is the central judgment. Inference proceeds by applying typing rules recursively to subterms, using unification and generalization to compute principal types.